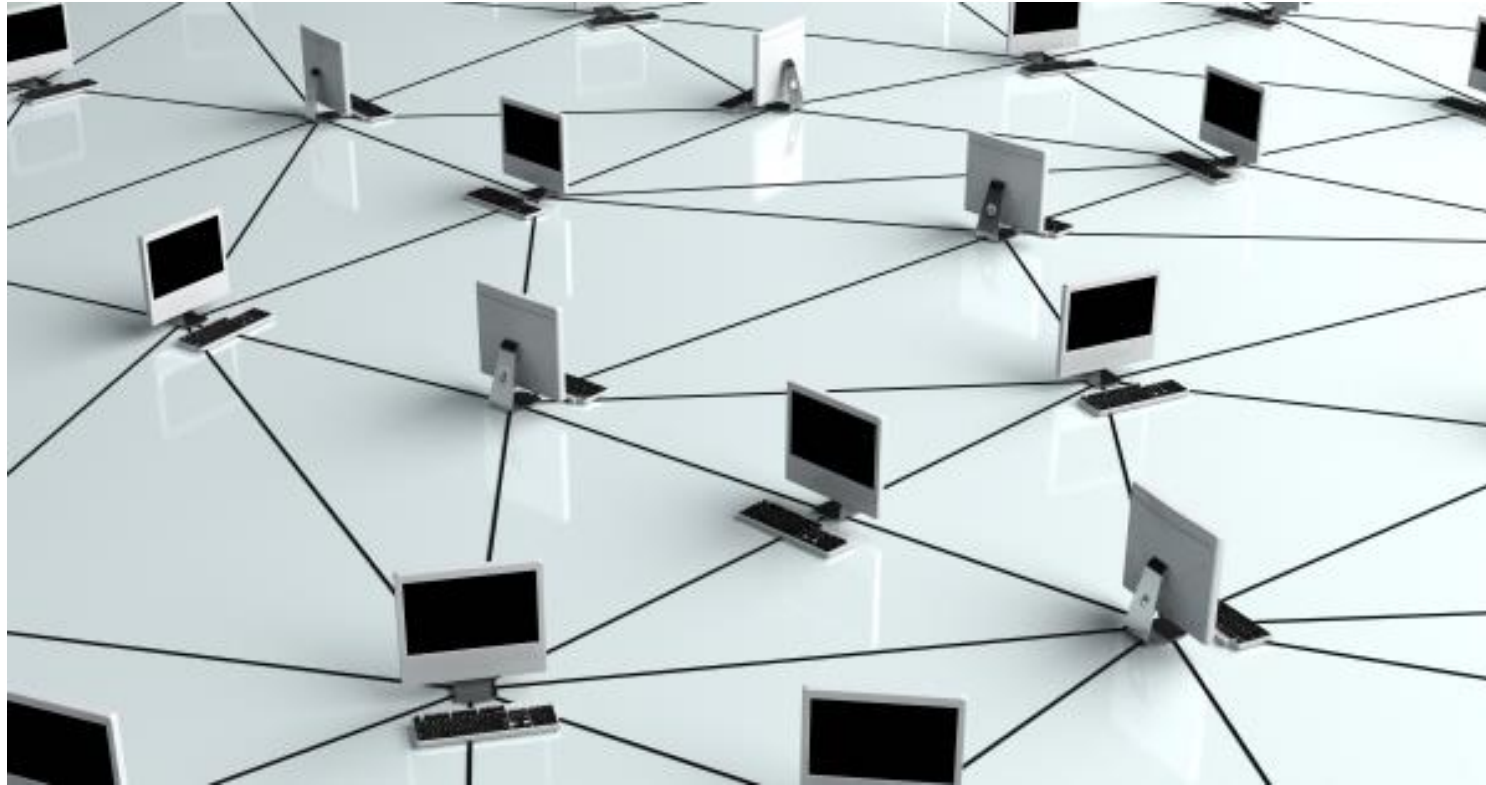
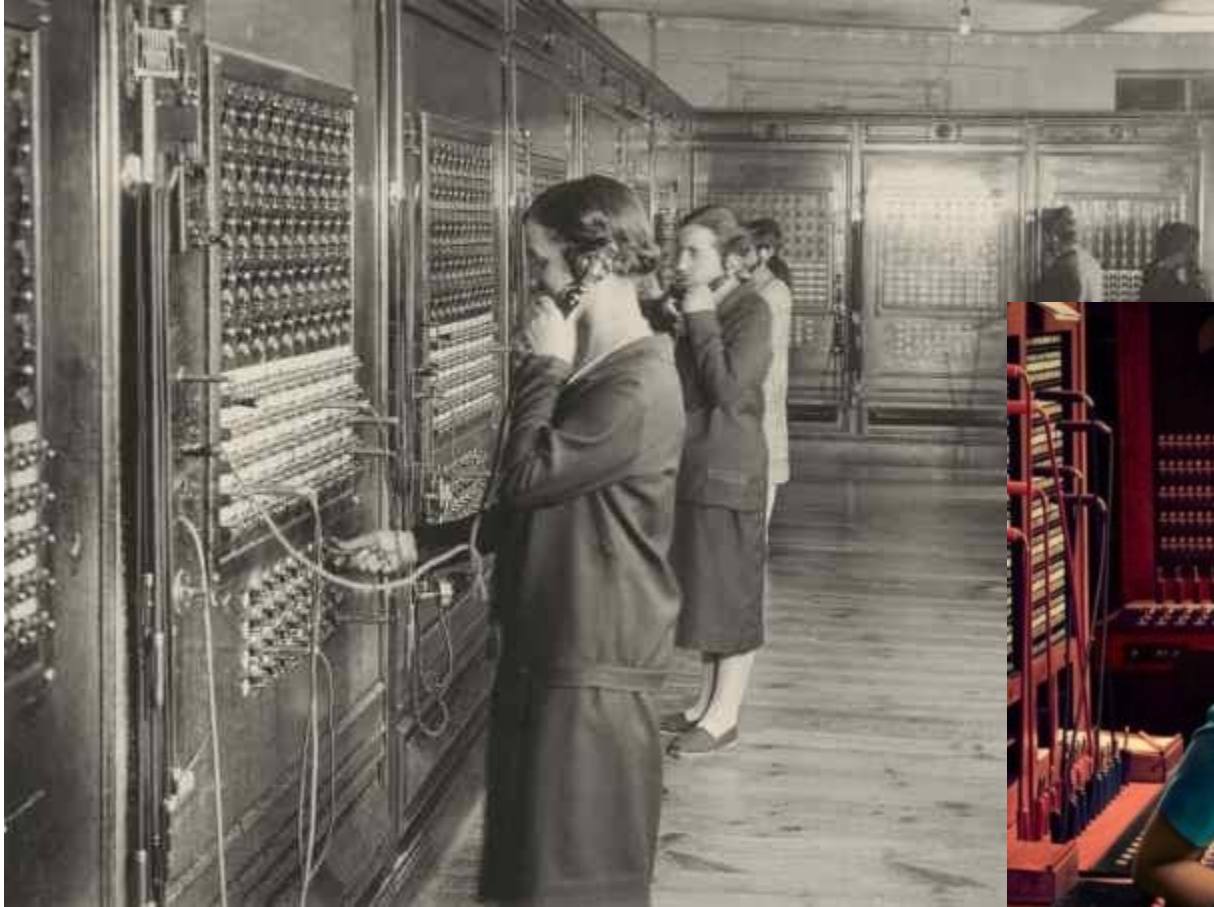


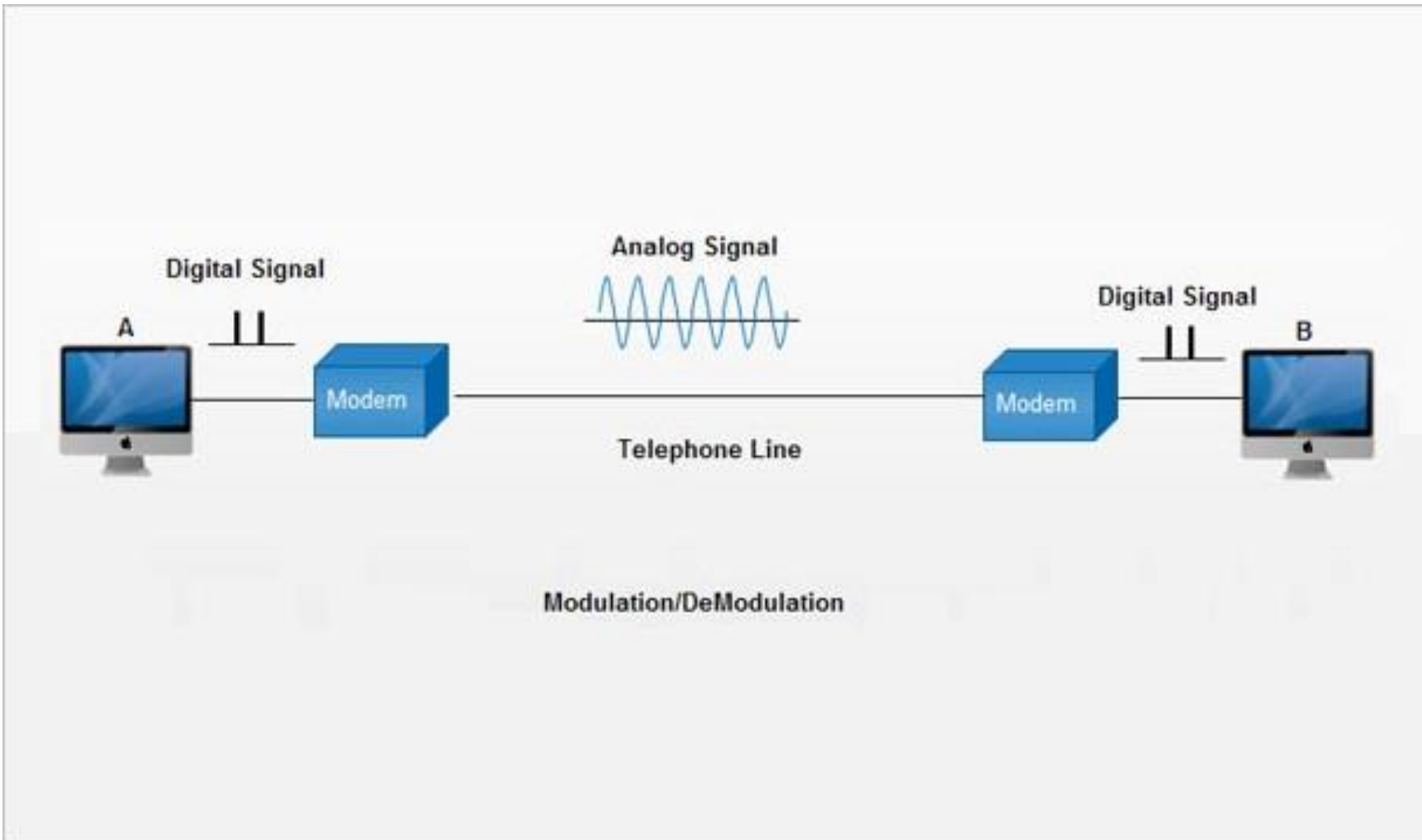
Introducción a redes



Cómo hablan los ordenadores

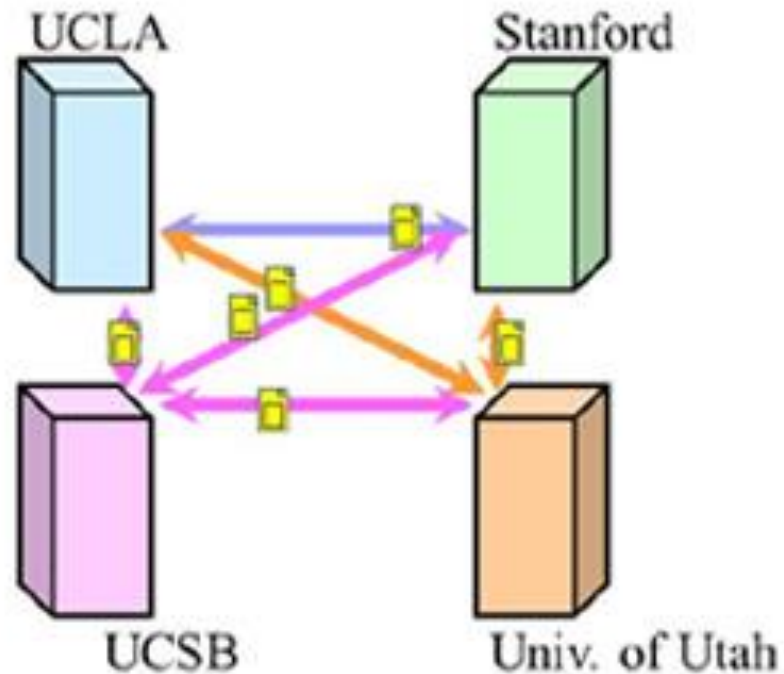


Cómo hablan los ordenadores



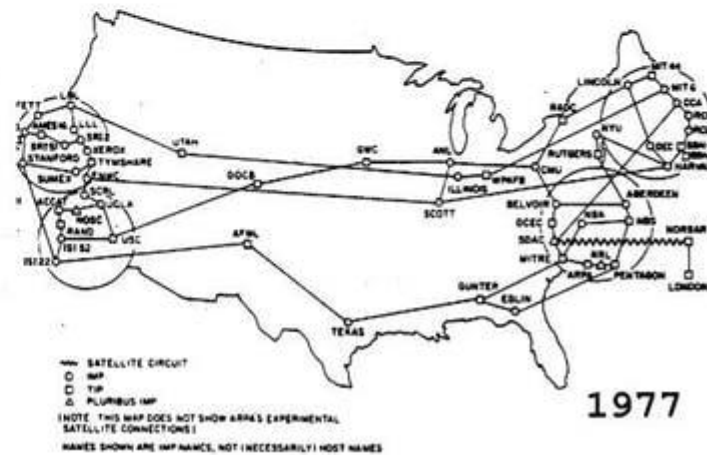
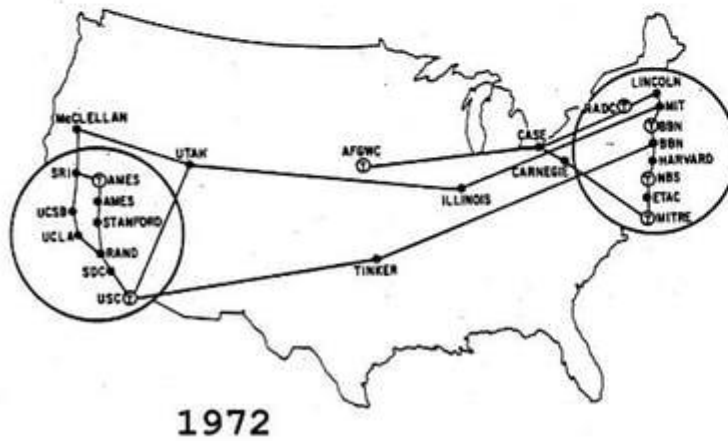
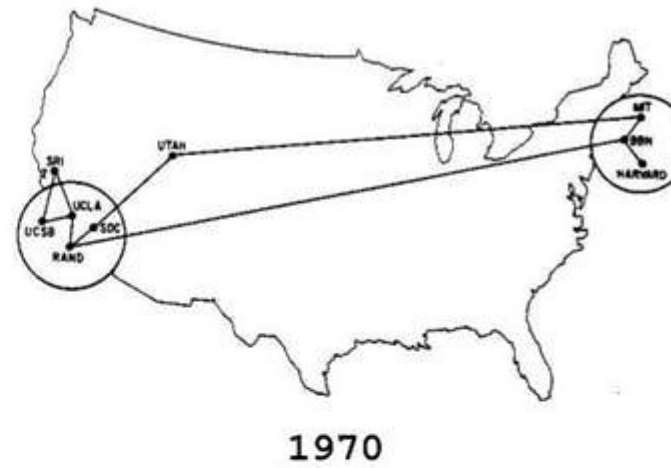
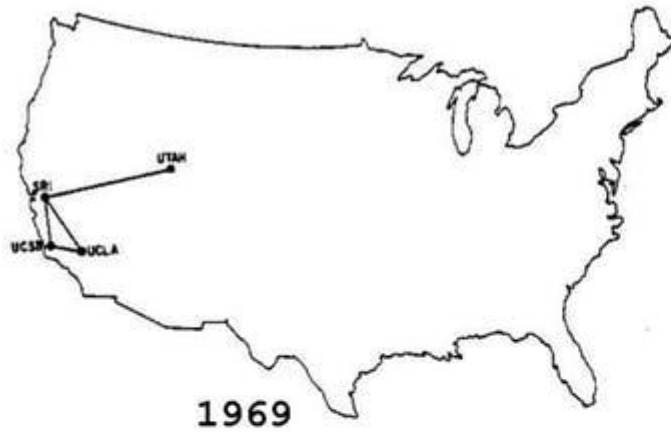
Cómo nació Internet

1969: The original ARPANET had 4 nodes

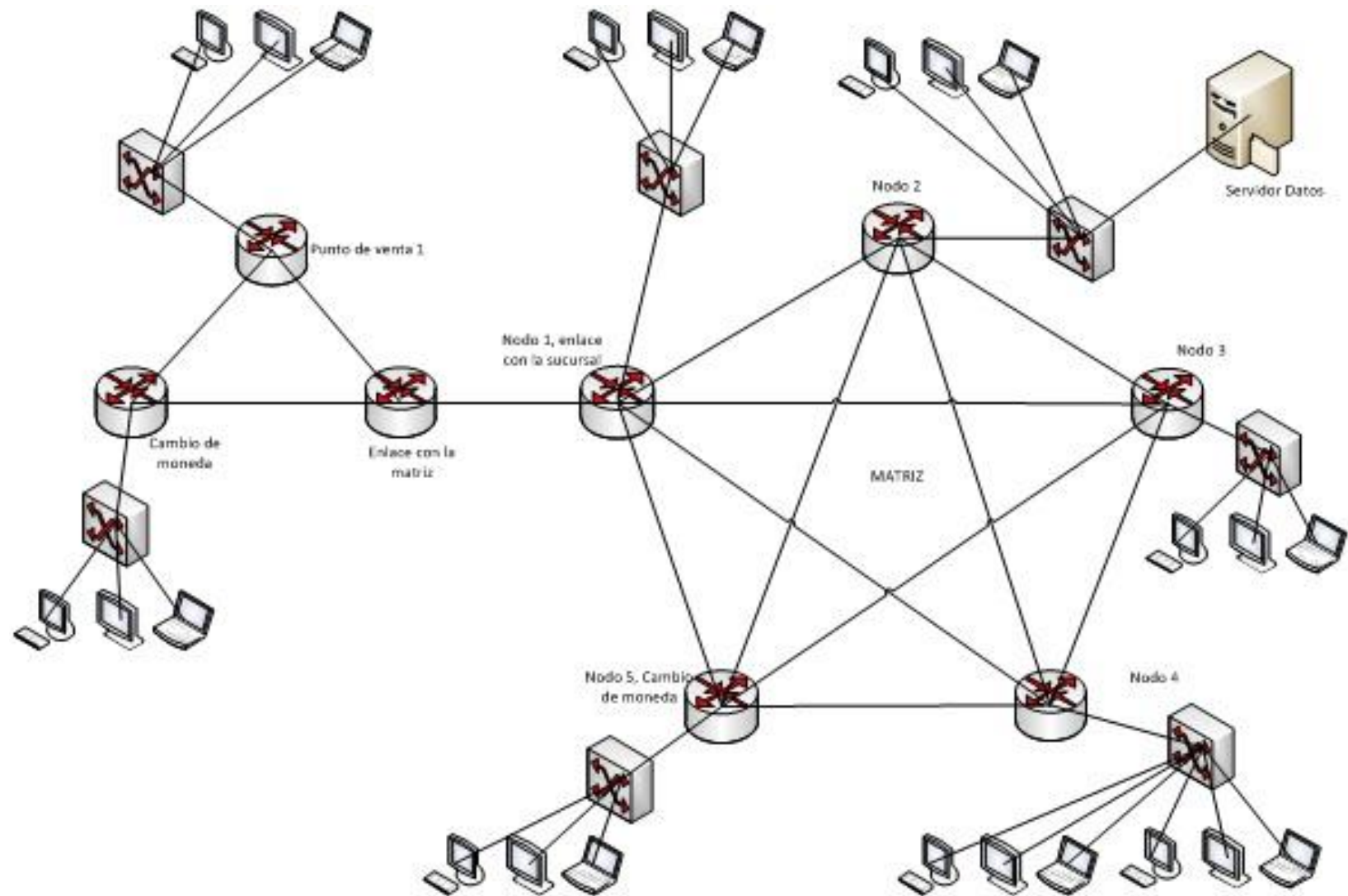


A message could start out at UCSB take many different routes to get to Stanford

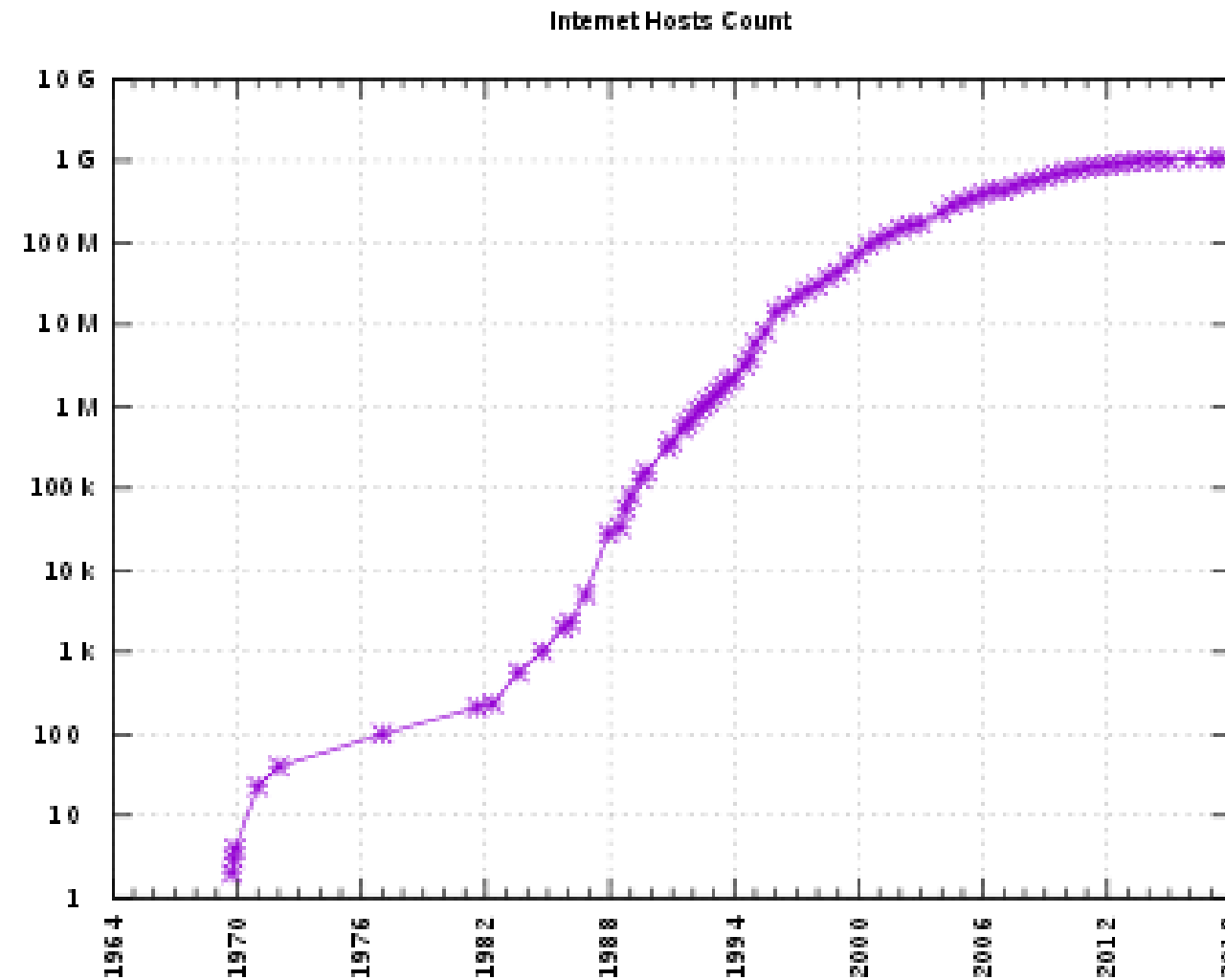
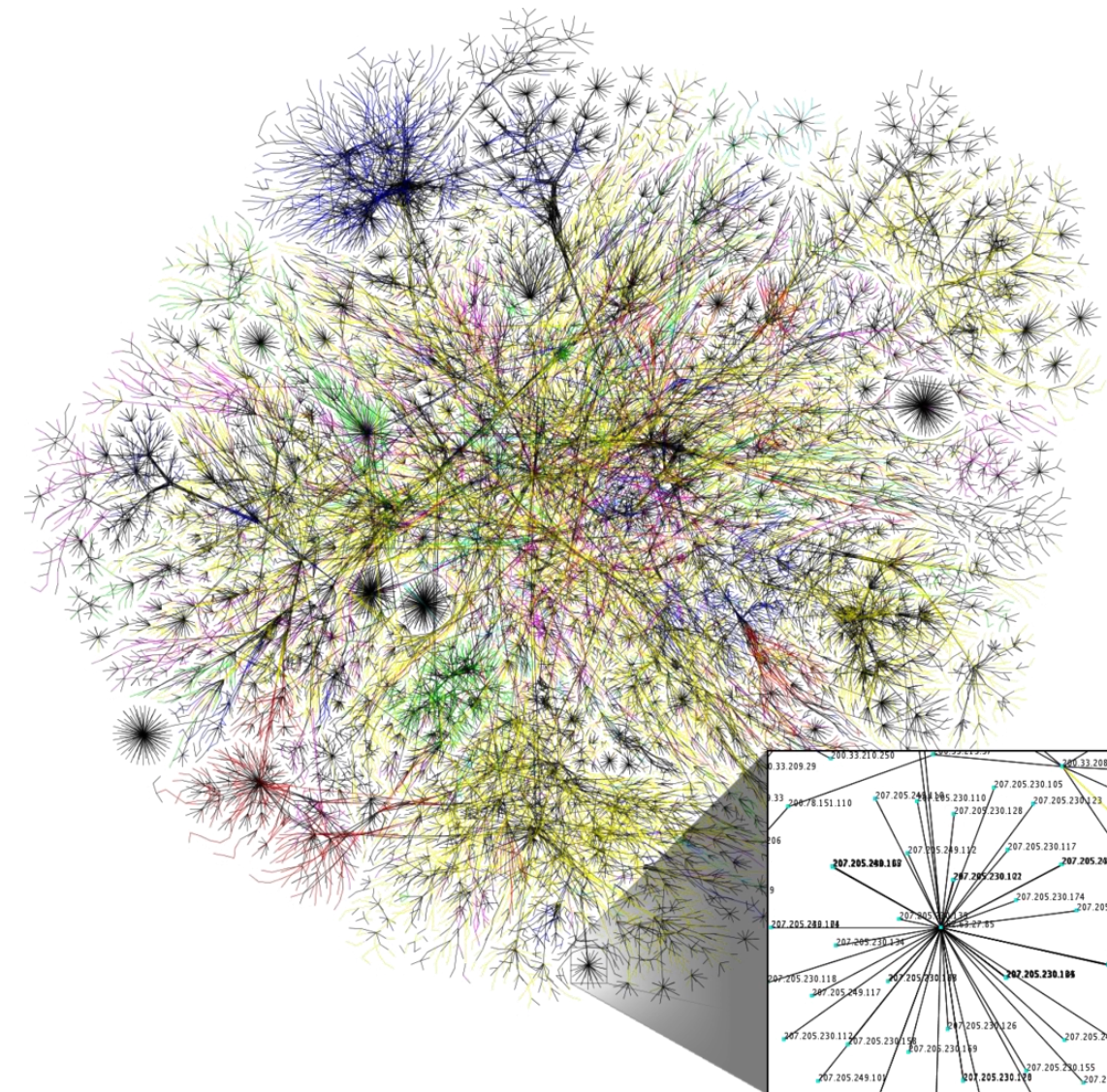
Cómo nació Internet



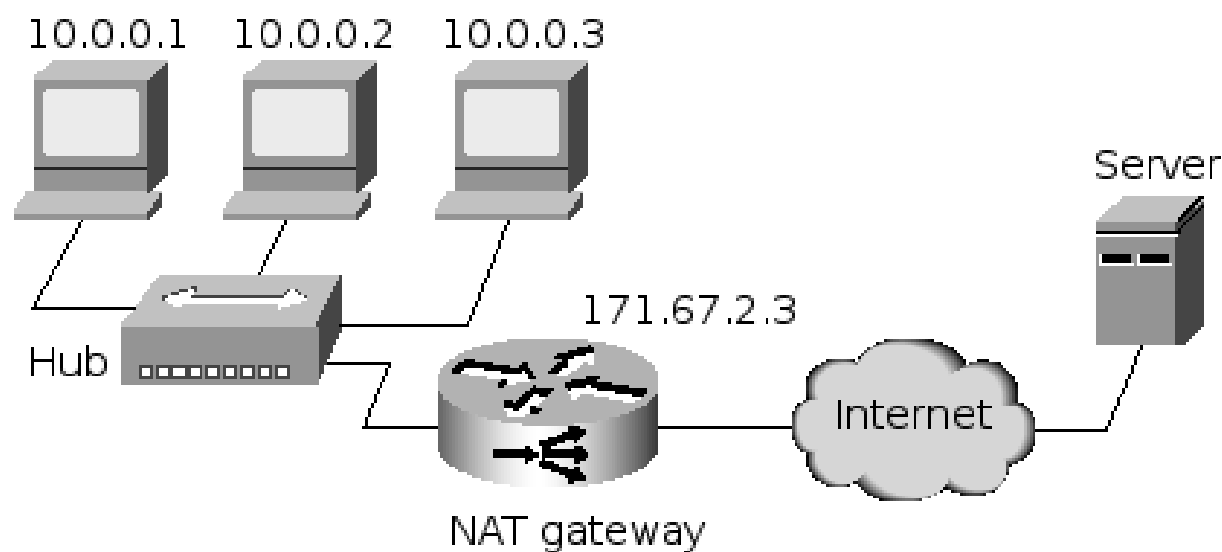
Internet conecta redes



Tamaño de Internet



Direcciones IP



An IPv4 address (dotted-decimal notation)

172 . 16 . 254 . 1

↓ ↓ ↓ ↓
10101100 .00010000 .11111110 .00000001

1 byte=8 bits

32 bits (4 x 8), or 4 bytes

¿Qué IP tiene tu ordenador?

¿y tu móvil?

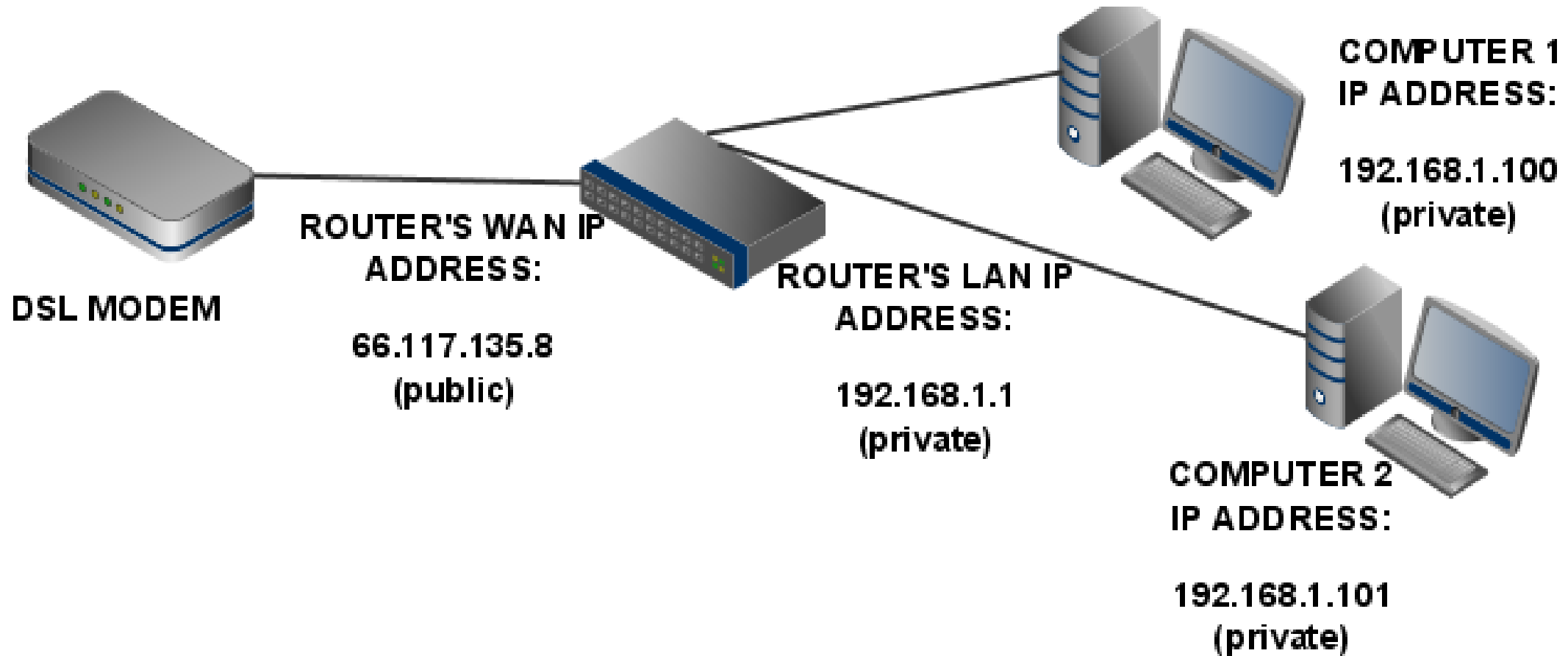
¿y tu Televisor?

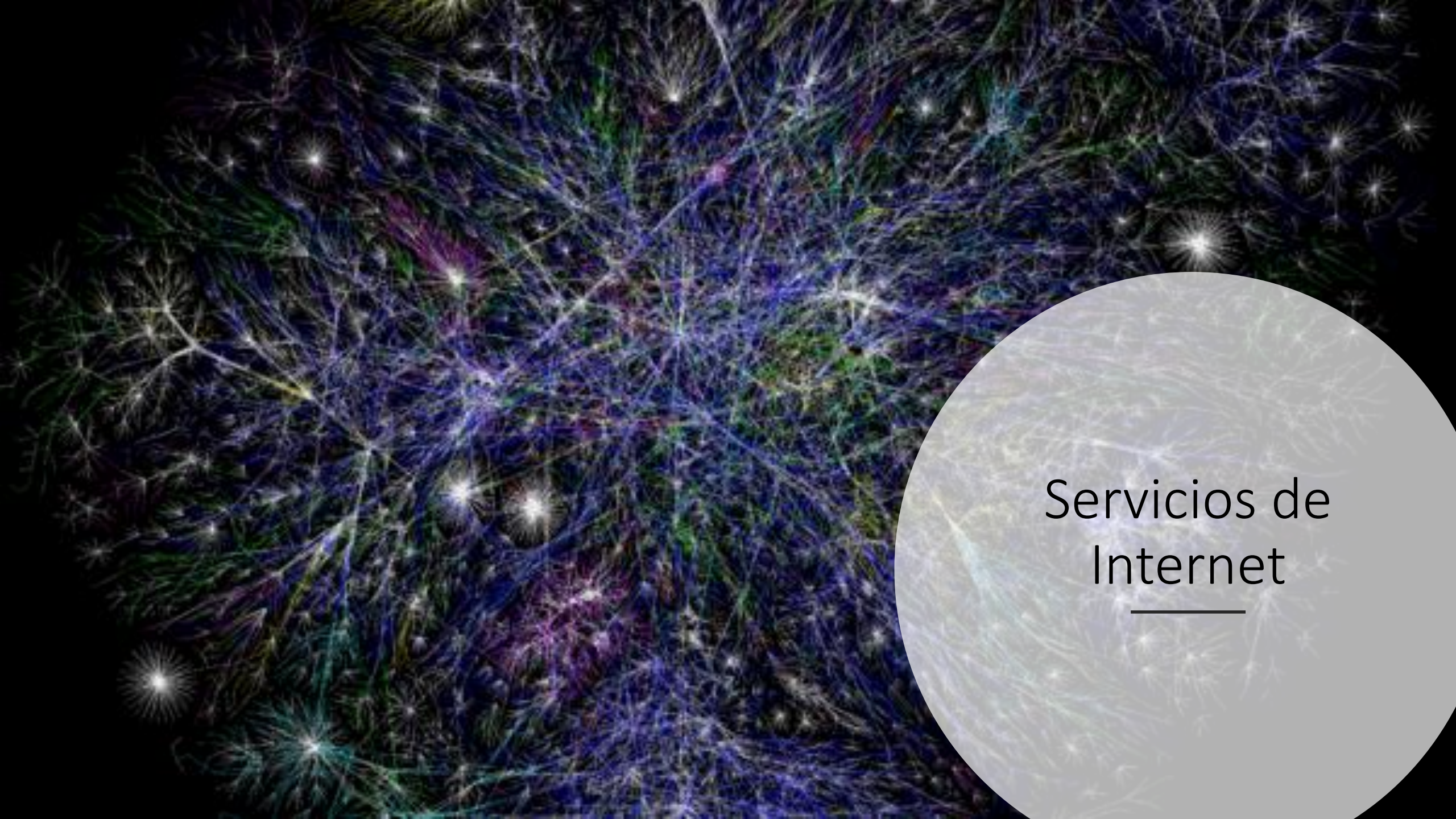
127.0.0.1

172.16.0.9

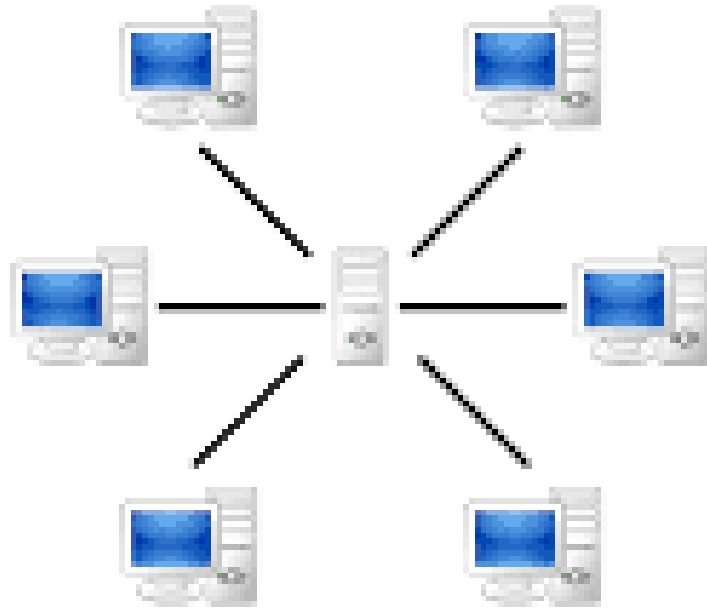
192.0.0.7

Direcciones IP públicas y privadas

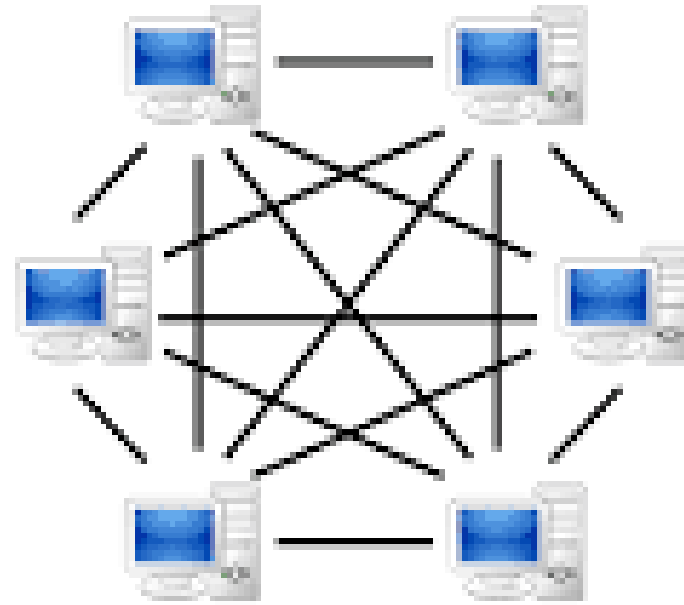




Servicios de Internet



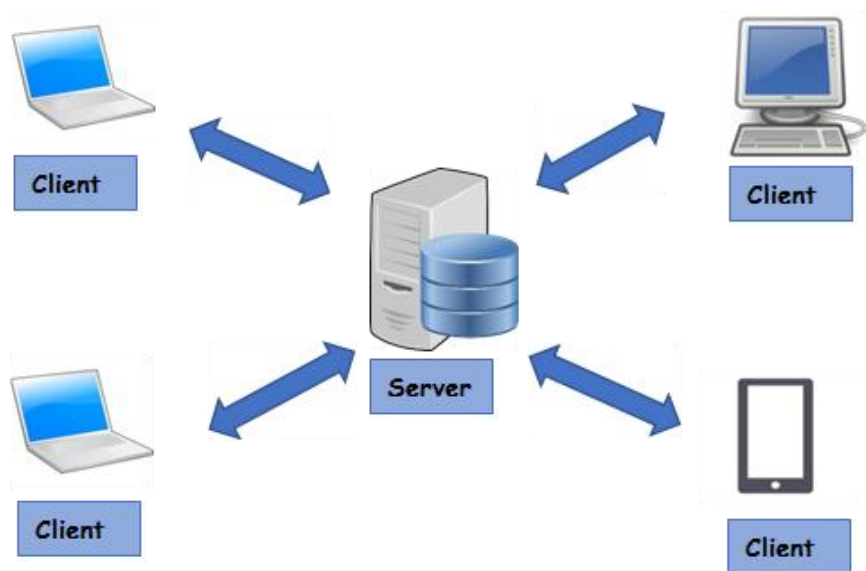
Server-based



P2P-network

P2P vs Cliente Servidor

Modelo Cliente-Servidor



Un ordenador “central” ofrece servicios a clientes que se conectan a él: es su servidor.

Tipos de servidores: web, ficheros, streaming, noticias ...

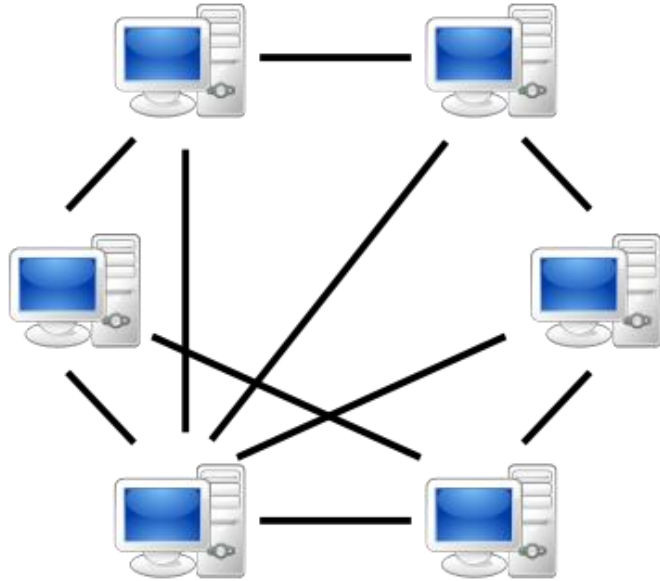
El servidor necesita ser un ordenador más potente para poder atender a todos los potenciales clientes.

El servicio es fácil de gestionar: se hace de forma centralizada. Es más fácil de controlar.

Punto único de fallo: si se cae adiós al servicio para todos los clientes.

La escalabilidad del servicio es un problema: todos los clientes acceden al mismo servidor.

Modelo P2P



<https://upload.wikimedia.org/wikipedia/commons/thumb/3/3f/P2P-network.svg/464px-P2P-network.svg.png>

Todos los clientes se conectan entre sí para implementar el servicio. Las comunicaciones son directas entre los clientes

¿Cómo localizar los clientes?

No hay punto único de fallo

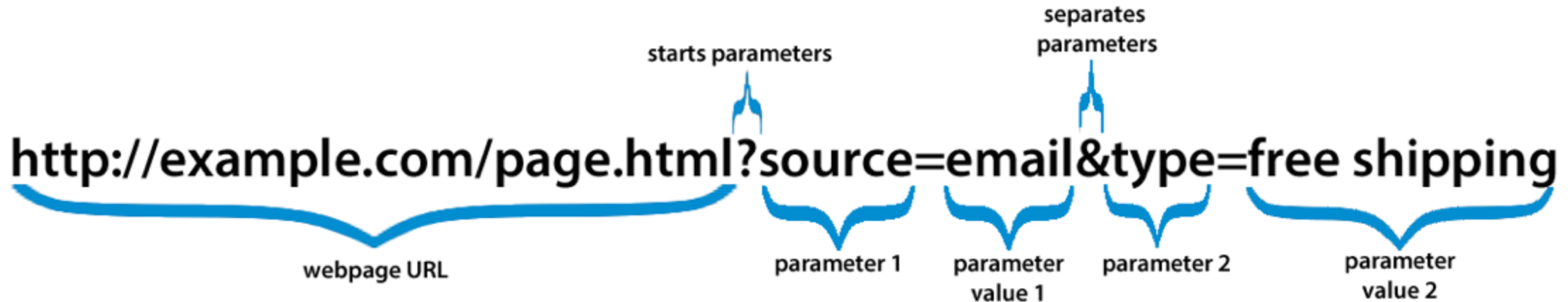
La escalabilidad del servicio es mayor: cada nuevo cliente aporta nuevos recursos.

Mayor privacidad: no hay registro en un servidor central.

Servicios más innovadores usan P2P: Bitcoin (blockchain como ejemplo).

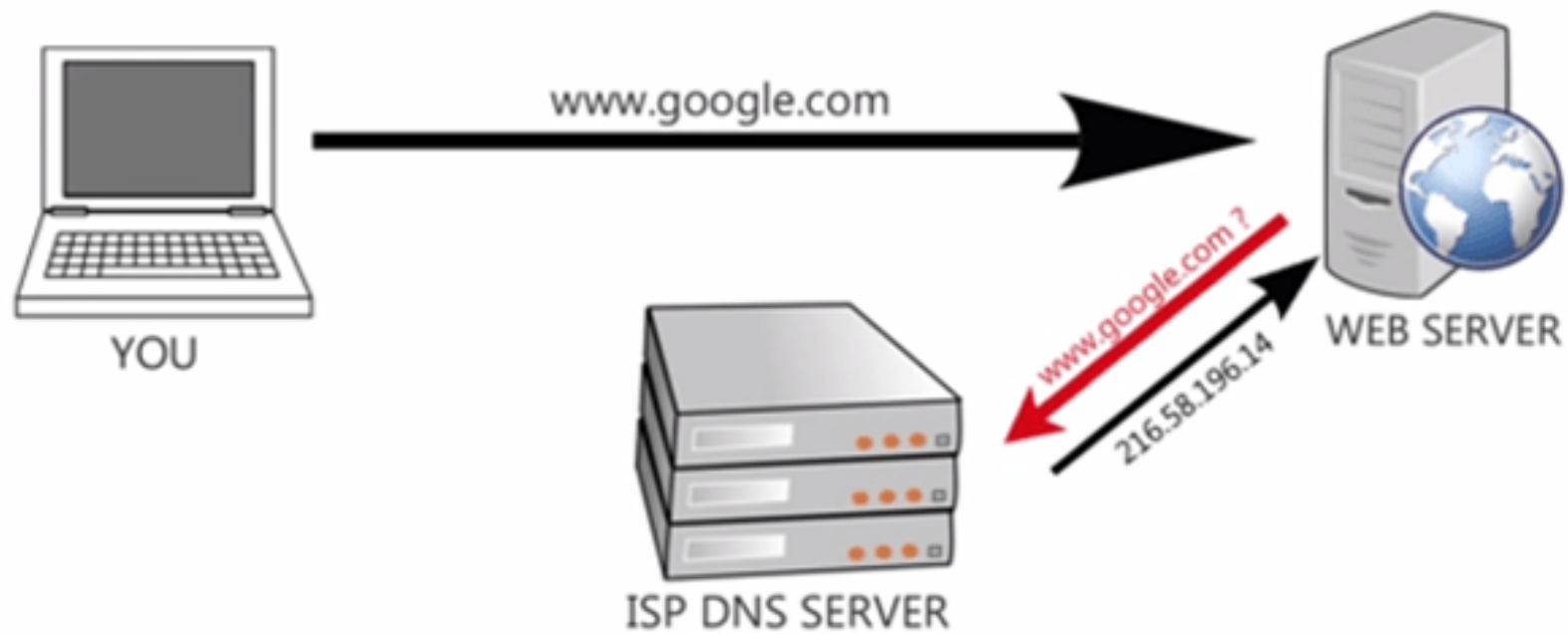
Modelo más complejo de gestionar (SLA)

URL



Servidor de DNS

HOW DNS WORKS



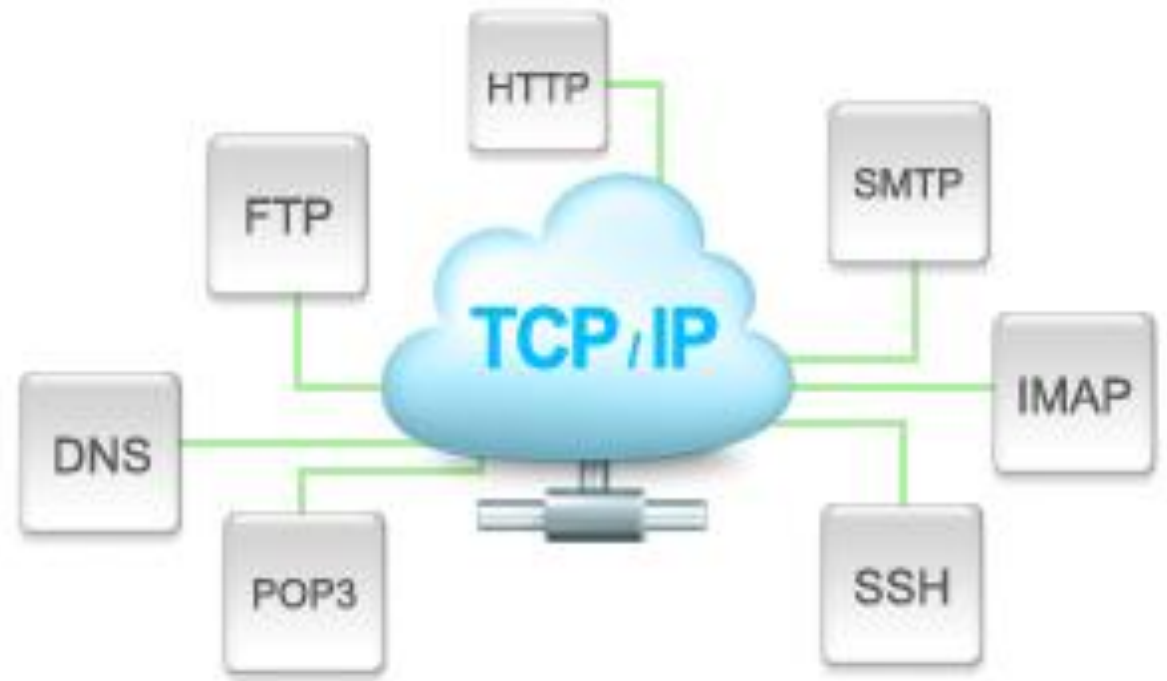
Protocolos

HTTP

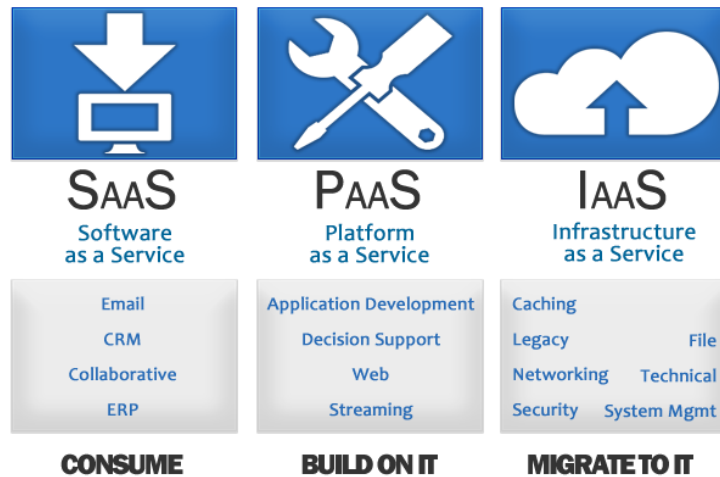
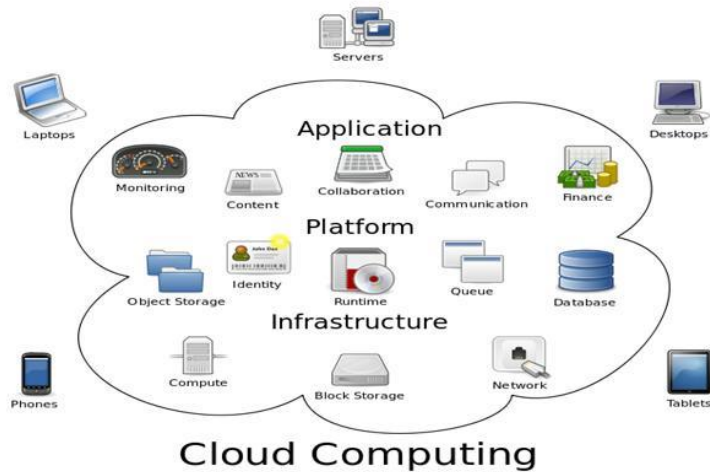
ⓘ <http://domain-name.com>

HTTPS

🔒 Secure | <https://domain.com>

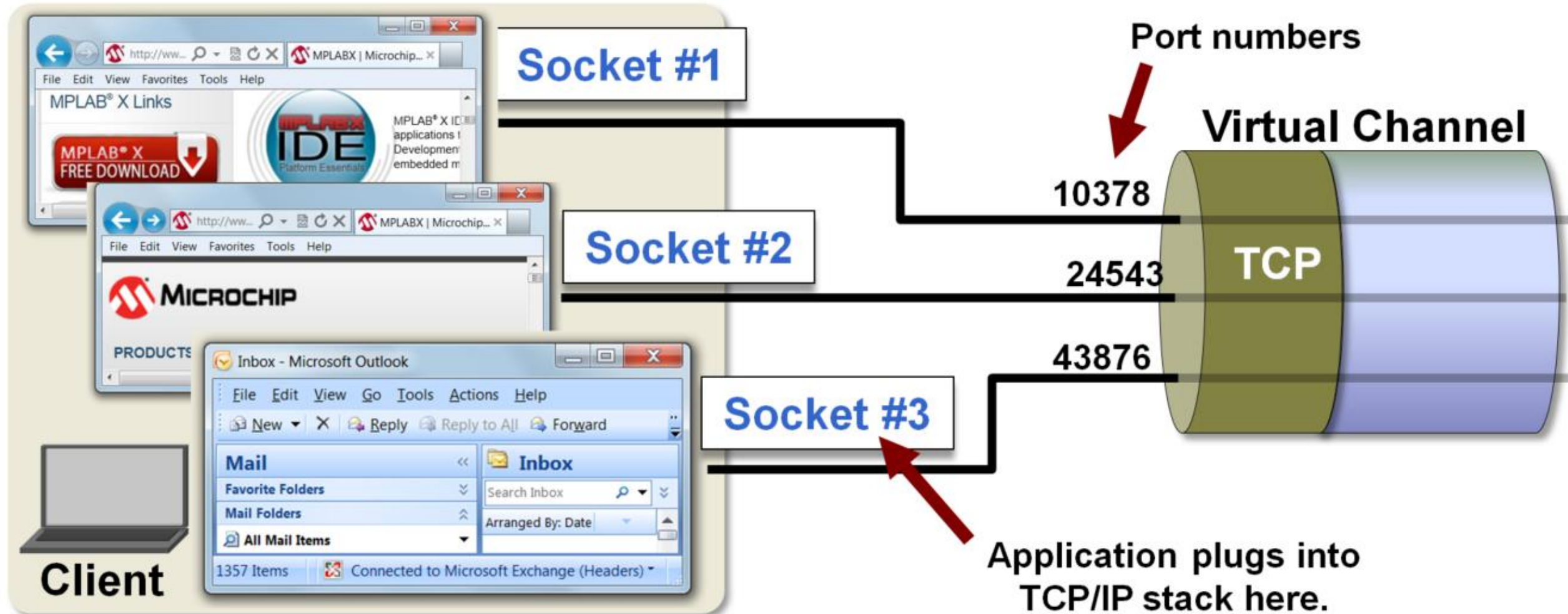


Cloud Computing

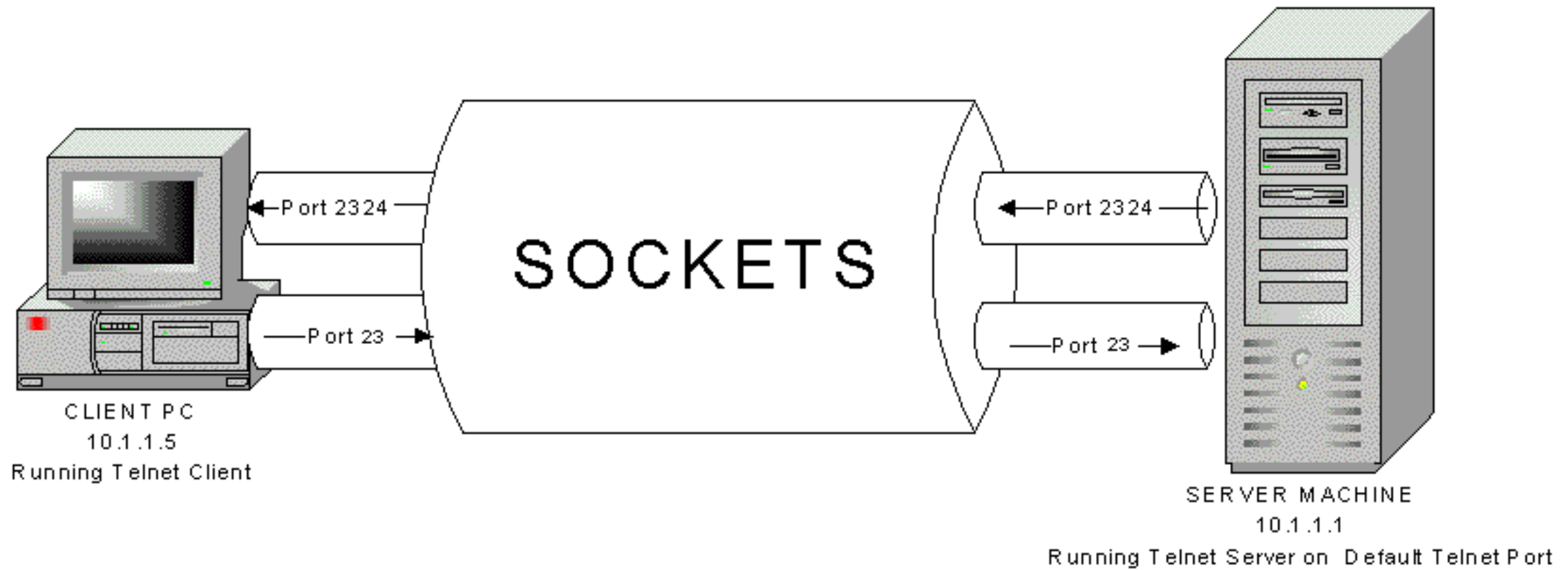


- Más que un modelo de red es una forma de desplegar los diferentes modelos de servicios de red gestionar tanto los ordenadores como los servicios que forman parte modelo de servicio de red.
- Hay tres tipos de Cloud Computing principales que se basan en ofrecer como servicio (as a Service) diferentes capas.
- **IaaS:** los ordenadores y la red (Infrastructure) están en la nube. Podemos acceder a ellos pero sin acceso físico y en ocasiones, sin saber dónde están.
- **PaaS:** lo que se ofrece como servicio es una plataforma (Python por ejemplo) donde poder desplegar nuestros desarrollos. Es el modelo que usan los desarrolladores de software.
- **SaaS:** lo que se ofrece como servicio es el propio software final, las funcionalidades que ofrece. Por ejemplo, Google ofrece el servicio de búsqueda.

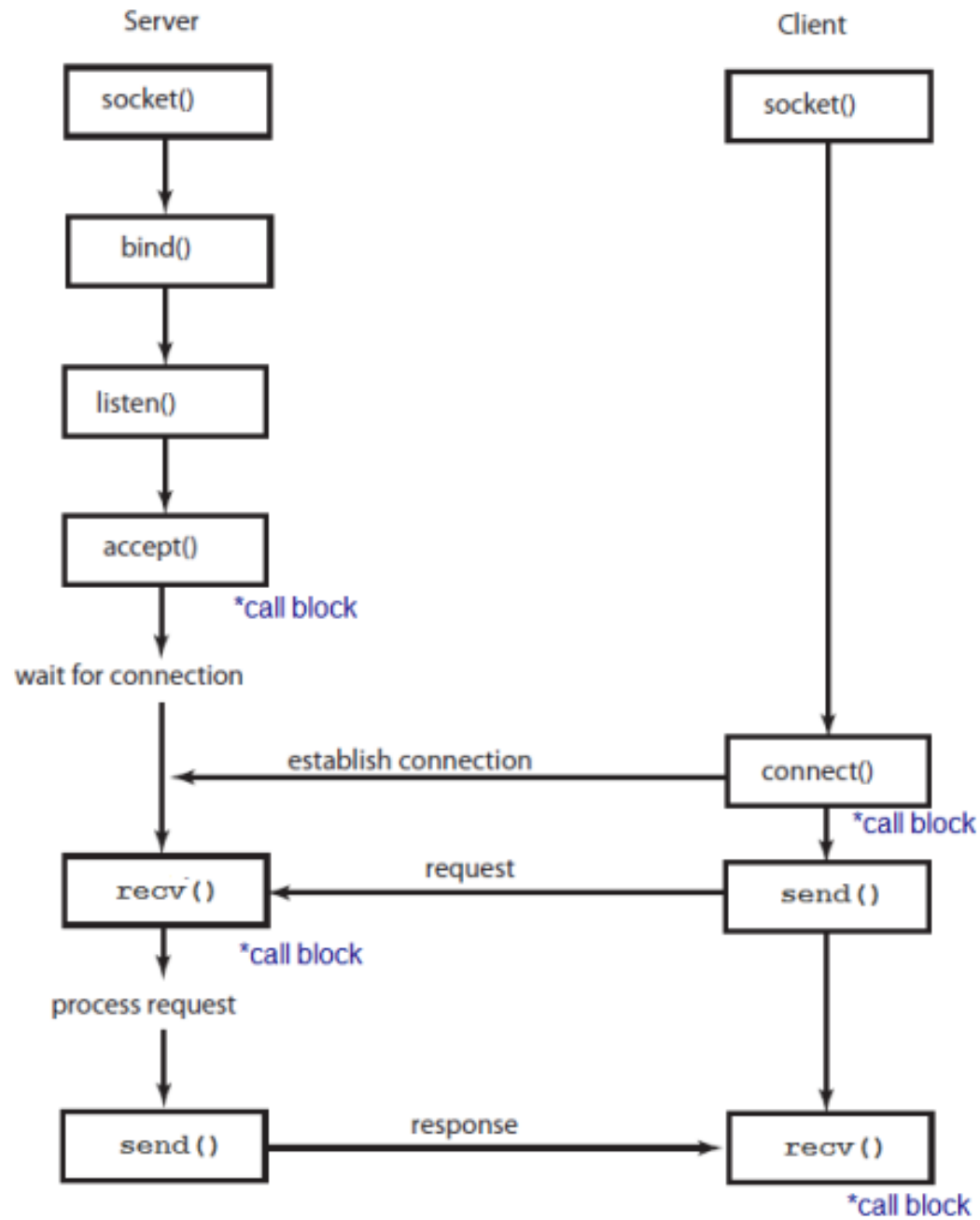
Puertos



Sockets



Programando sockets

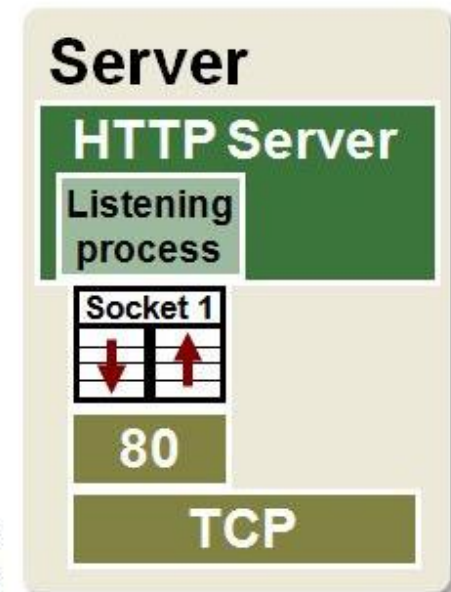


El servidor crea el socket y escucha

Server Sockets	Socket 1	
Transport	TCP	
Source Port	80	
Source IP Addr	192.168.1.102	
Destination Port	n/a	
Destination IP Addr	n/a	

```
# create an INET, STREAMing socket
serversocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# bind the socket to a public host, and a well-known port
serversocket.bind((socket.gethostname(), 80))
# become a server socket
serversocket.listen(5)
```

IP Address =
192.168.1.102



El cliente crea el socket y conecta

Transport	TCP
Source Port	10378
Source IP Addr	192.168.1.101
Destination Port	80
Destination IP Addr	192.168.1.102



```
# create an INET, STREAMing socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# now connect to the web server on port 80 - the normal http
port
s.connect(("www.urjc.es", 80))
```

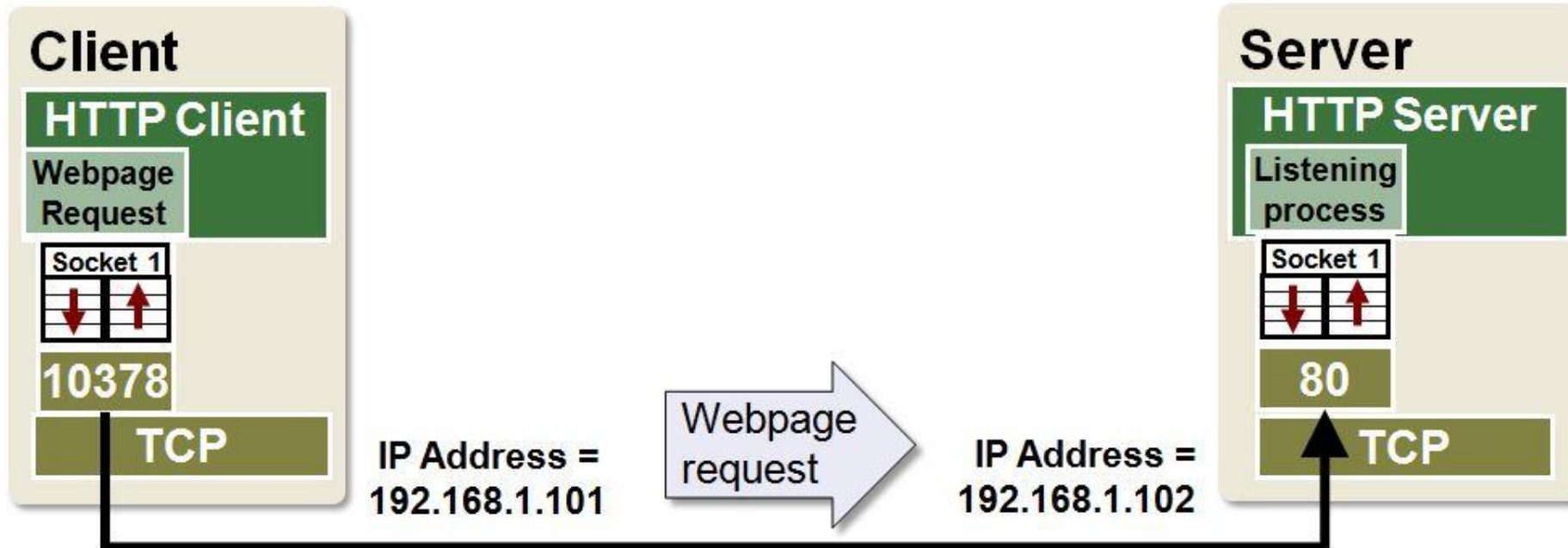
IP Address =
192.168.1.101

<http://microchipdeveloper.com/tcpip:use-sockets-to-establish-a-tcp-connection>

Se establece la conexión

Client Sockets	Socket 1
Transport	TCP
Source Port	10378
Source IP Addr	192.168.1.101
Destination Port	80
Destination IP Addr	192.168.1.102

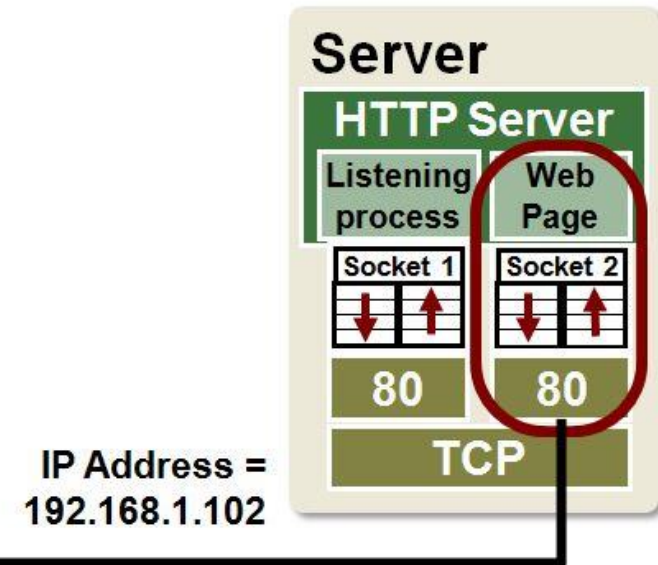
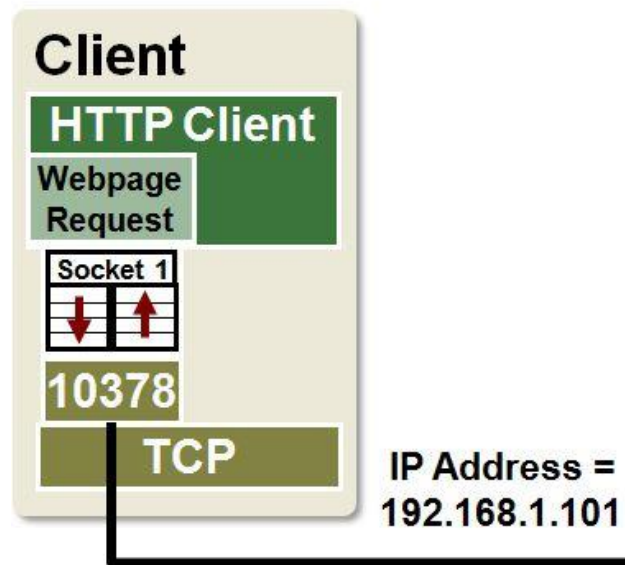
Server Sockets	Socket 1	
Transport	TCP	
Source Port	80	
Source IP Addr	192.168.1.102	
Destination Port	n/a	
Destination IP Addr	n/a	



La conexión queda establecida

Client Sockets	Socket 1
Transport	TCP
Source Port	10378
Source IP Addr	192.168.1.101
Destination Port	80
Destination IP Addr	192.168.1.102

Server Sockets	Socket 1	Socket 2
Transport	TCP	TCP
Source Port	80	80
Source IP Addr	192.168.1.102	192.168.1.102
Destination Port	n/a	10378
Destination IP Addr	n/a	192.168.1.101

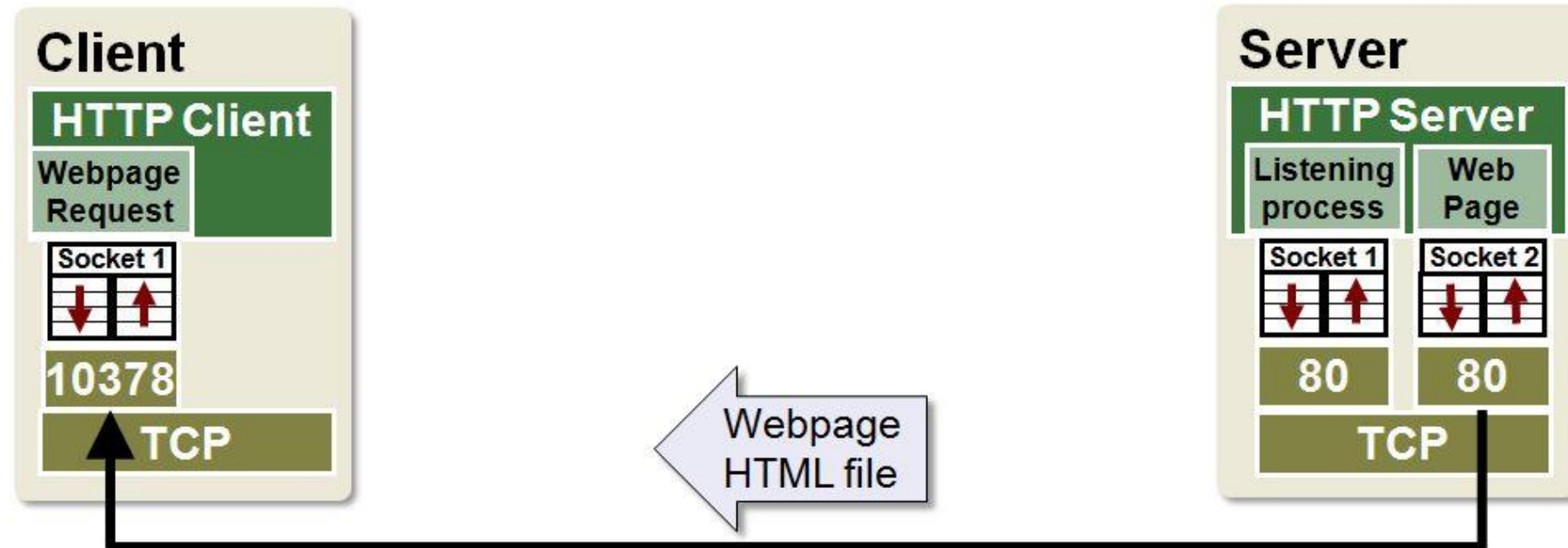


```
connection, client_address = serversocket.accept()
```


Se intercambian datos

Client Sockets	Socket 1
Transport	TCP
Source Port	10378
Source IP Addr	192.168.1.101
Destination Port	80
Destination IP Addr	192.168.1.102

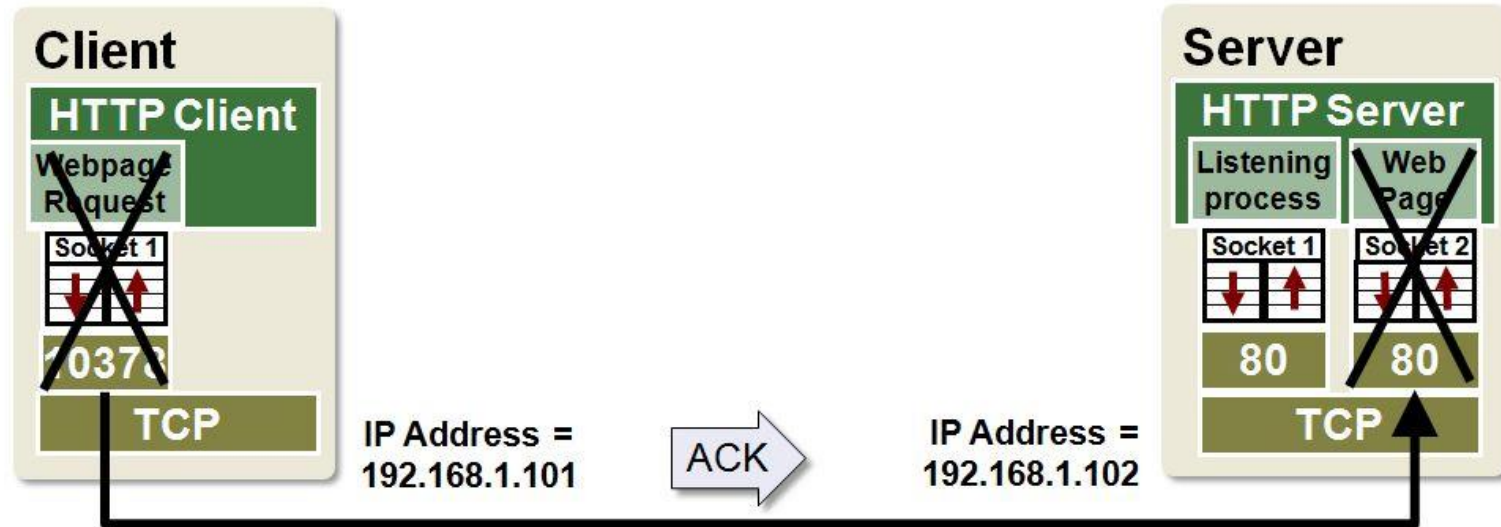
Server Sockets	Socket 1	Socket 2
Transport	TCP	TCP
Source Port	80	80
Source IP Addr	192.168.1.102	192.168.1.102
Destination Port	n/a	10378
Destination IP Addr	n/a	192.168.1.101



Se cierra la conexión

Client Sockets	Socket 1
Transport	TCP
Source Port	10378
Source IP Addr	192.168.1.101
Destination Port	80
Destination IP Addr	192.168.1.102

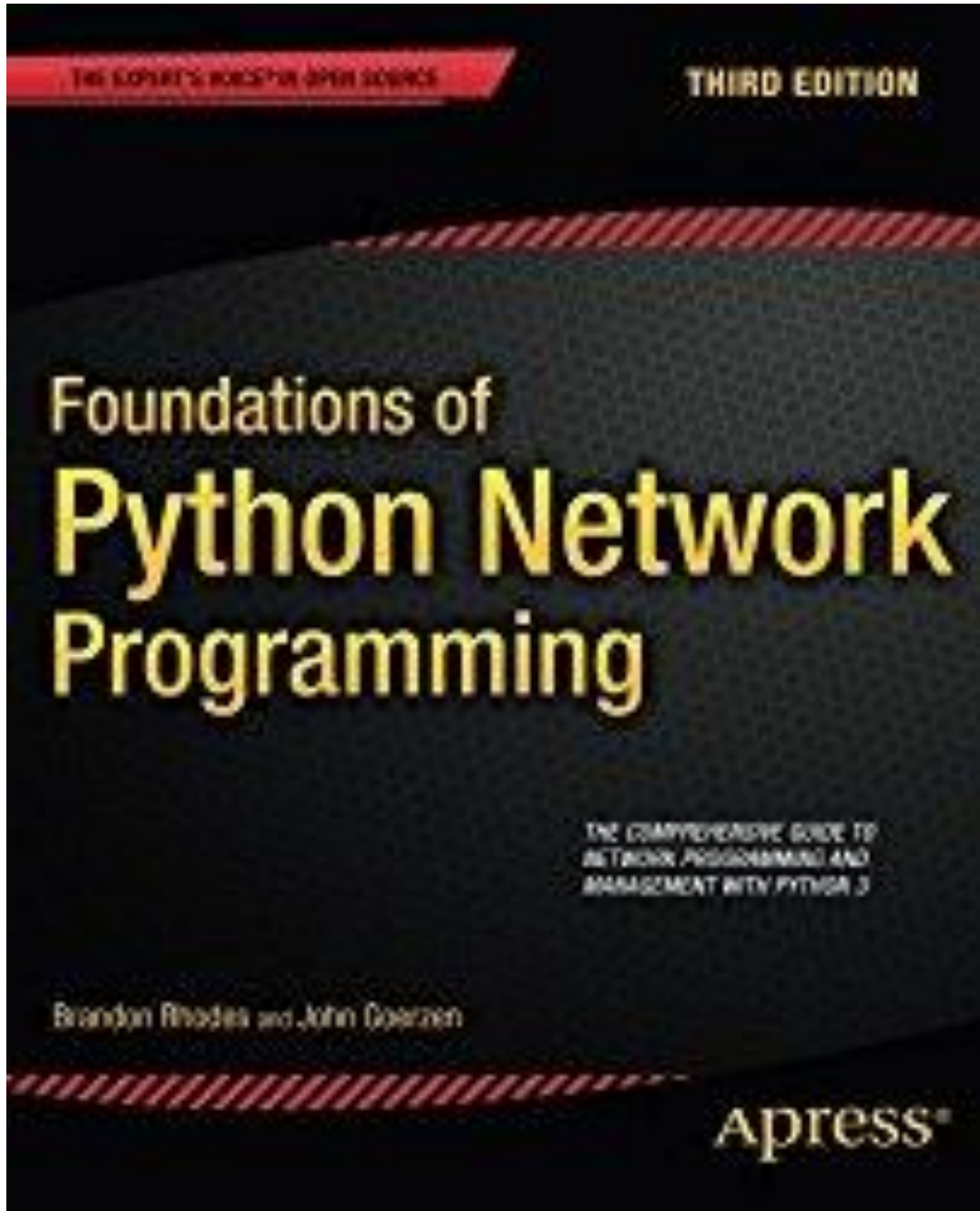
Server Sockets	Socket 1	Socket 2
Transport	TCP	TCP
Source Port	80	80
Source IP Addr	192.168.1.102	192.168.1.102
Destination Port	n/a	10378
Destination IP Addr	n/a	192.168.1.101



Probad estos tutoriales

<http://www.pythondiario.com/2015/01/simple-programa-clienteservidor-socket.html>

<http://aprendepythonvolando.blogspot.com.es/2012/06/tutorial-18-sockets-en-python.html>



Foundations of Python Network Programming

John Goerzen

Foundations of Python Network Programming (Apress) — scripts and examples

567 commits 1 branch 1 release 3 contributors

Branches: master New pull request Find file Clone or download

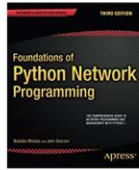
Latest commit 9536289 on Aug 14, 2017

diagrams	Try clarifying that mod_wsgi starts its own daemon	3 years ago
playground	Fix #16 by making link to extended notes	a year ago
py2	Add comments atop requirements.txt files	3 years ago
py3	Replace "IN" module import with inlined constants	6 months ago
gitignore	Update documentation for new released Vagrant box	3 years ago
README.md	Make link to network playground more obvious	a year ago

Foundations of Python Network Programming

Welcome!

This GitHub repository offers all of the example Python code from the Third Edition of *Foundations of Python Network Programming* as revised by [Brandon Rhodes](#) for Python 3:



From Amazon.com (affiliate link)
From Apress (the publisher)

Each chapter's source code lives in its own directory:

- Chapter 1: Introduction to Client-Server Networking

Vocabulario

- Antes de cerrar el tema, asegúrate que conoces los conceptos:
 - Red de ordenadores
 - Internet
 - Protocolo de comunicaciones
 - Dirección IP y puerto
 - DNS
 - Modelo Cliente-Servidor
 - Modelo Peer-to-peer o P2P
 - Computación en la nube
 - URL, Uniform Resource Locator
 - Socket
 - Programación con socket TCP: servidor y cliente

Temas para discutir

- TCP/IP e Internet sirven para crear redes de ordenador. ¿Qué alternativas han existido o podrían existir en el futuro?
- Los nombres de domino funcionan de forma jerárquica. ¿Existen otros servicios en internet similares? ¿Se te ocurren alternativas no sean jerárquicas?
- La URL es válida para el modelo cliente-servidor. ¿Serviría para un modelo P2P?
- Los números IP tradicionales (IPv4) se acabaron hace tiempo. ¿Qué alternativas hay?
- ¿Por qué existen tantos protocolos?
- Hace poco se anunció que Spotify abandonaba su modelo P2P. ¿Por qué utilizo dicho modelo? ¿Qué razones crees que hay para que ahora lo abandone?
- Algunos de los servicios más famosos podrían estar o no en la nube. De algunos de éstos (Dropbox, Gmail, Netflix, Moodle, la web de la universidad, un juego de PC, un juego flash, torrent) discute si crees que están en la nube o no. ¿Cómo los diseñarías tu?
- El ejemplo de cliente-servidor que hemos programado es muy básico. ¿Qué limitaciones crees que tiene?